

Project Documentation

Date	29 September 2026
Team ID	SWTID-2026-3976
Project Name	AI FAQ Assistant API
Maximum Marks	20 Marks

1. Team Details

S.No	Name	Role	Email
1	V. Sabarivasan	Team Lead	sabarivasan287@gmail.com
2	Yogeshkumar N	Member	Yogeshsparrow72@gmail.com
3	Karthika D	Member	karthikaok610@gmail.com
4	Uma Sankari G	Member	hyntaff@gmail.com
5	Vishal Frankline L	Member	Frankclg333@gmail.com

Team ID	SWTID-2026-3976
Project Title	AI FAQ Assistant – AI-Powered FAQ & Customer Support Automation Backend
GitHub Repository	https://github.com/sabarivasan-2105/AI_FAQ_Assistant_API-
Demo / API Testing Video	Demo recording (Google Drive)

2. Project Description

The AI FAQ Assistant is a robust, secure, and intelligent REST API platform designed to streamline FAQ creation, user authentication, and AI-powered content management. Built with Node.js and Express.js, the application uses MongoDB as its primary data store along with Mongoose ODM for schema validation and data modeling. Security is central to the architecture: JWT (JSON Web Tokens) protect every write route, and bcrypt secures every stored password. The backend handles automated FAQ generation and keyword-based FAQ search, alongside centralized error handling that returns consistent, sanitized JSON error responses instead of leaking raw database errors.

3. Scenario-Based Case Study

Persona: Priya, a customer support operations manager who handles a large backlog of repetitive user inquiries but struggles to organize entries, classify topics by category, write precise answers, and keep the FAQ set current. Manually drafting complex support answers is time-consuming, and there is no unified permission layer protecting one author's FAQ entries from being changed by someone else. With the AI FAQ Assistant, Priya (or any authenticated creator) can register, request an AI-generated answer or draft FAQ pair from Google Gemini, save it as a categorized FAQ she owns, and let any public consumer search the resulting collection by keyword – work that used to mean manually drafting and filing every entry now takes a single authenticated API call.

4. Software & Hardware Requirements

Category	Requirement
Operating System	Windows 10/11, macOS, or Linux (cross-platform)
Runtime	Node.js v18 or above
Package Manager	npm v9 or above
Framework	Express.js (v4/v5 ecosystem)
Database	MongoDB with Mongoose ODM
AI Service	Google Gemini API (<code>@google/genai</code>)
Development Tools	Visual Studio Code, Postman / Thunder Client, Git & GitHub
Minimum Hardware	Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent, 8 GB RAM (16 GB recommended), 1 GB available disk space

EPIC 1 - System Architecture

Incoming requests undergo filter processing (middleware) before interacting with data services. **Interface:** API tools like Postman query distinct route paths to extract JSON entities. **Express Server Gateway:** monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handles CORS headers. **Authentication Middleware:** extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths. **Granular Services (Logic Modules):** enforce isolated domain rules for user session life cycles and operational query modifications. **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries. **AI Flow (Service Integration):** connected to Google Gemini via SDK integration, letting the controller layer offload question-answering and automated FAQ-generation workflows.

Epic-1: Project Architecture

AI FAQ Assistant API – Technical Architecture

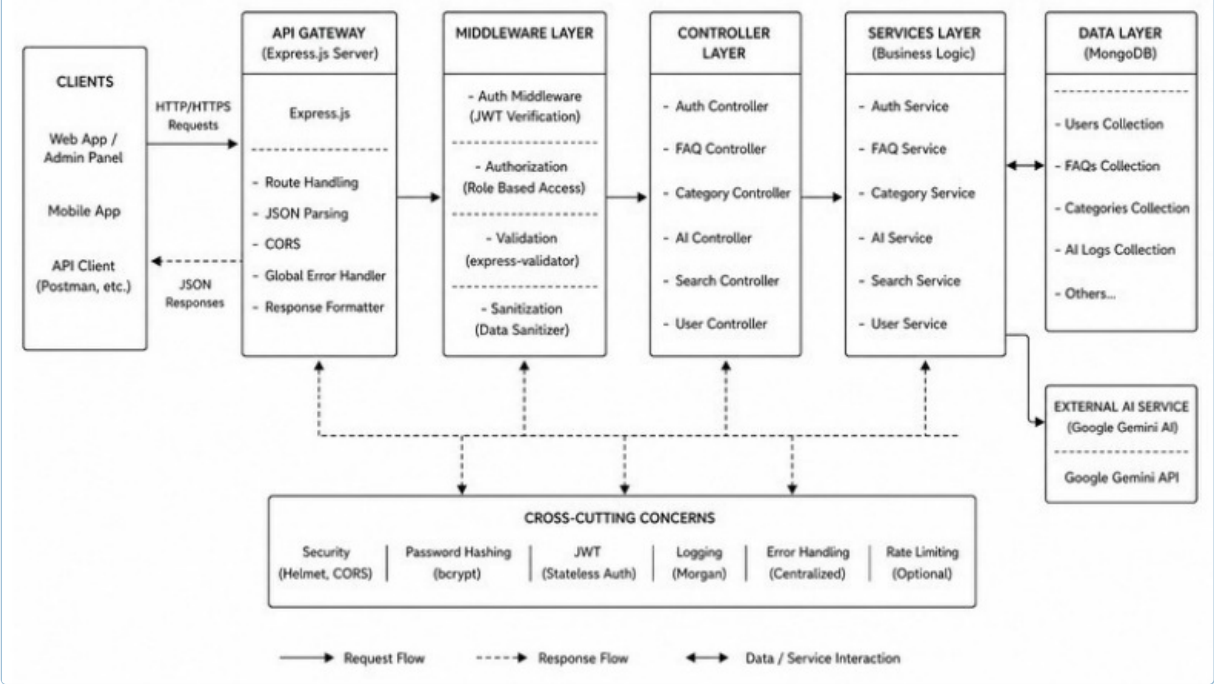
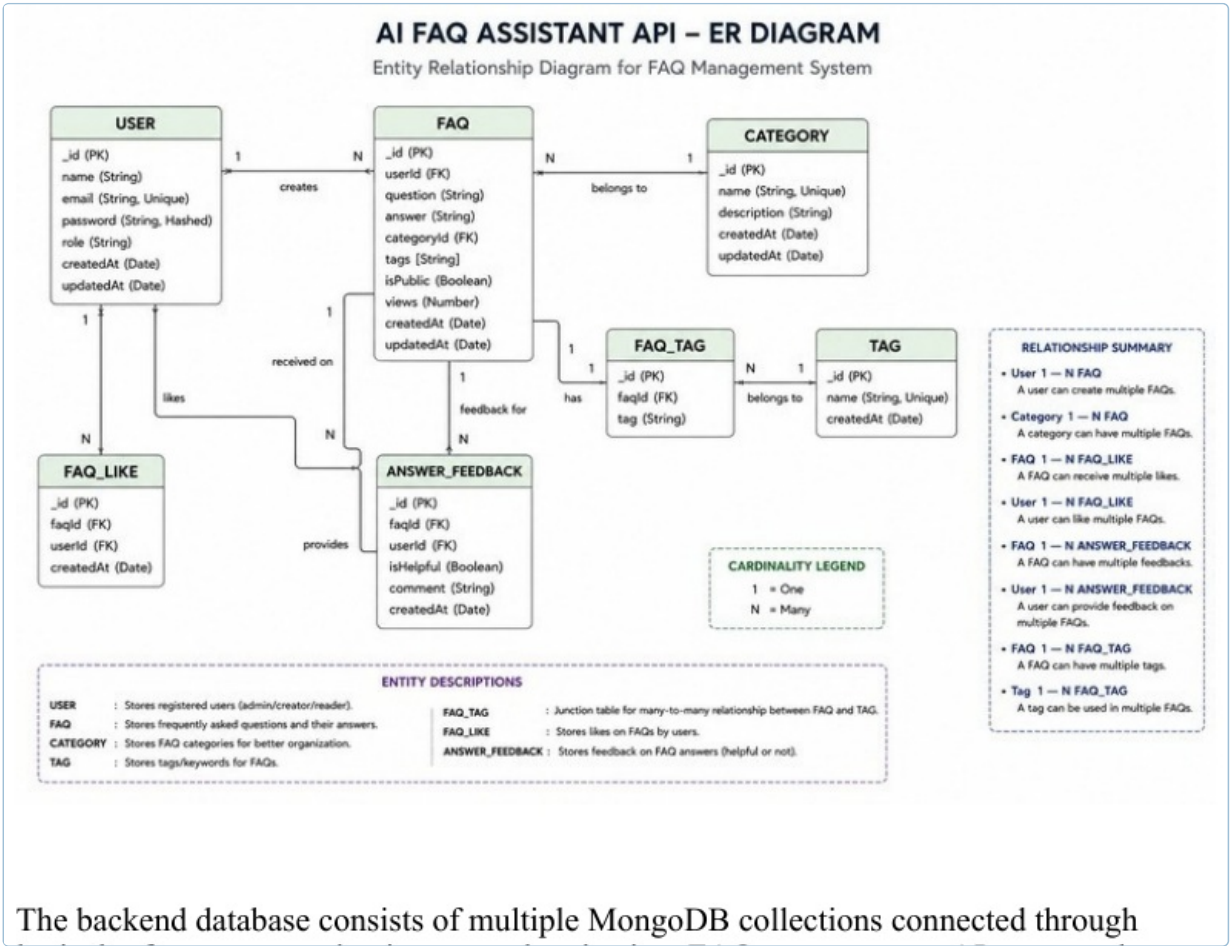


Fig. 1 – AI FAQ Assistant API technical architecture, as designed by the team: Clients → API Gateway → Middleware Layer → Controller Layer → Services Layer → Data Layer, with a parallel path from the Services Layer to the external Google Gemini AI service

ER Diagram - Data Model

Currently implemented model: two collections – **User** (`_id`, `name` [required], `email` [required, unique, lowercase], `password` [bcrypt-hashed, minlength 6], `timestamps`) and **FAQ** (`_id`, `question` [required], `answer` [required], `category` [required, enum: Technology/Education/Health/Banking/General, default General], `createdBy` [ObjectId reference to User, required], `timestamps`). There is no `role` field on the actual User schema and no separate Category, Tag, Like or Feedback collection – category is a validated enum string directly on FAQ.



The backend database consists of multiple MongoDB collections connected through

Fig. 2 – Full ER diagram as originally designed by the team: seven entities (User – including a `role` field, FAQ, Category, FAQ_TAG, TAG, FAQ_LIKE, ANSWER_FEEDBACK) with likes, tags and feedback as independent, queryable collections. The team implemented a simplified version of this design in this cycle – only User and FAQ exist as real Mongoose models, with category folded into a single enum field – and the fully normalized, richer structure shown here is recorded as the normalization target for the Scalability & Future Plan roadmap (see Phase 8).

Key Features

- JWT-based authentication protecting every FAQ-write and AI endpoint
- AI FAQ Generation: Google Gemini automatically drafts a question/answer pair from a topic
- AI Direct-Answer Generation: Google Gemini answers a free-form support question
- Keyword FAQ Search for fast information retrieval
- Complete CRUD operations on FAQ records, scoped to the authenticated creator
- Category organization via a validated enum field on each FAQ
- Password encryption with bcrypt before storage
- Centralized error handling for standardized, sanitized API error responses
- Mongoose schema validation ensuring data integrity before persistence
- RESTful, modular Express.js route architecture

Roles and Responsibilities

Role (as designed)	Capabilities (as designed)
Admin	Manage all registered users; view, update or delete any FAQ; monitor AI-generated content; manage FAQ categories; access complete system functionality
Content Creator	Create new FAQs; edit and delete personal FAQ records; generate FAQs using AI; organize FAQs under categories
Authenticated User	View public FAQs; search FAQs by keyword; generate AI-assisted answers; access personal profile information
Public User	View published FAQs; search FAQs; cannot create, edit or delete FAQ records; no access to protected endpoints

The team's User Flow diagram labels these same four tiers Admin / Editor / Author (Creator) / Reader (Consumer) - a naming difference from the table above with no effect on behavior. In the code actually shipped this cycle, the `User` model carries no `role` field and `authMiddleware.js` exposes only one guard, `protect` : every logged-in user can create, update and delete FAQs, and there is no separate Admin tier or draft/approval workflow yet. The four-role design above and in the diagrams is recorded as a Scalability & Future Plan target (see Phase 8).

User Flow

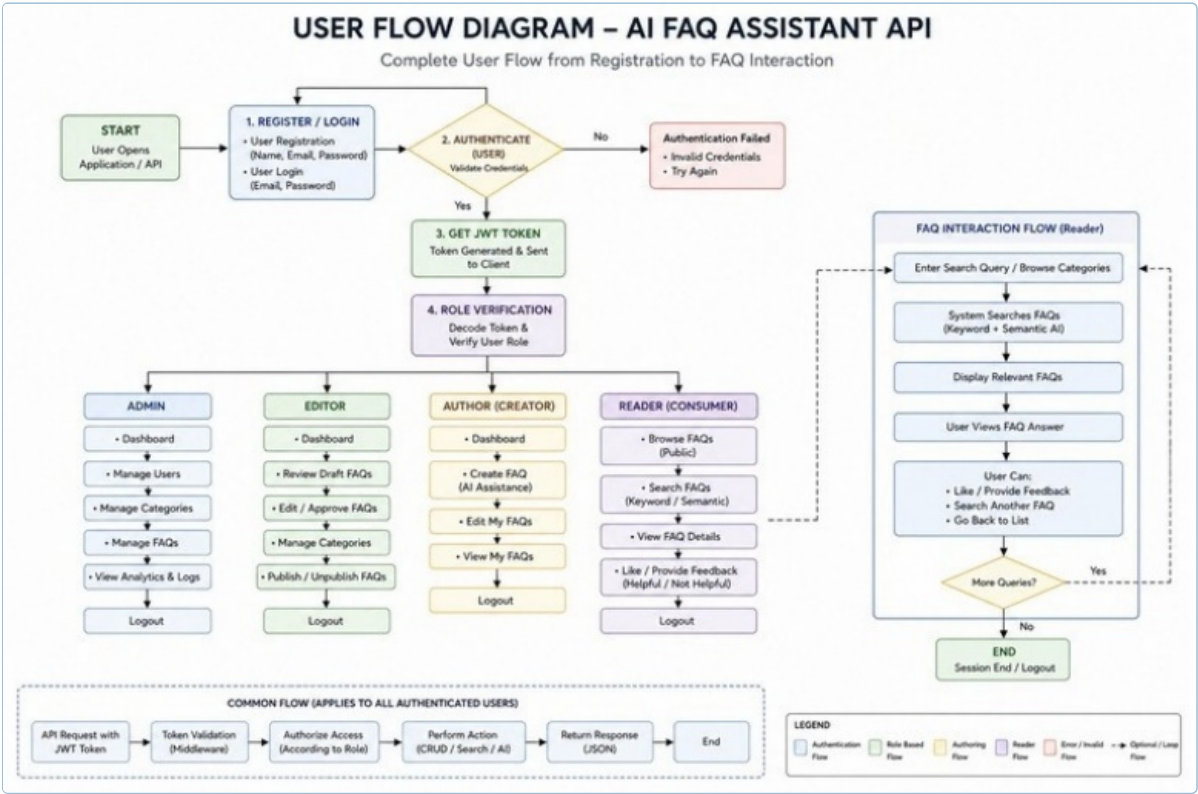


Fig. 3 - Register/Login → JWT issued → role verification → role-specific dashboard (Admin/Editor/Author/Reader) → FAQ interaction (search, view, like/feedback) as originally designed; the shipped backend implements the common authenticate → authorize (protect) → perform action → JSON response flow shown at the bottom of the diagram, without the role-specific branches above it

MVC Pattern

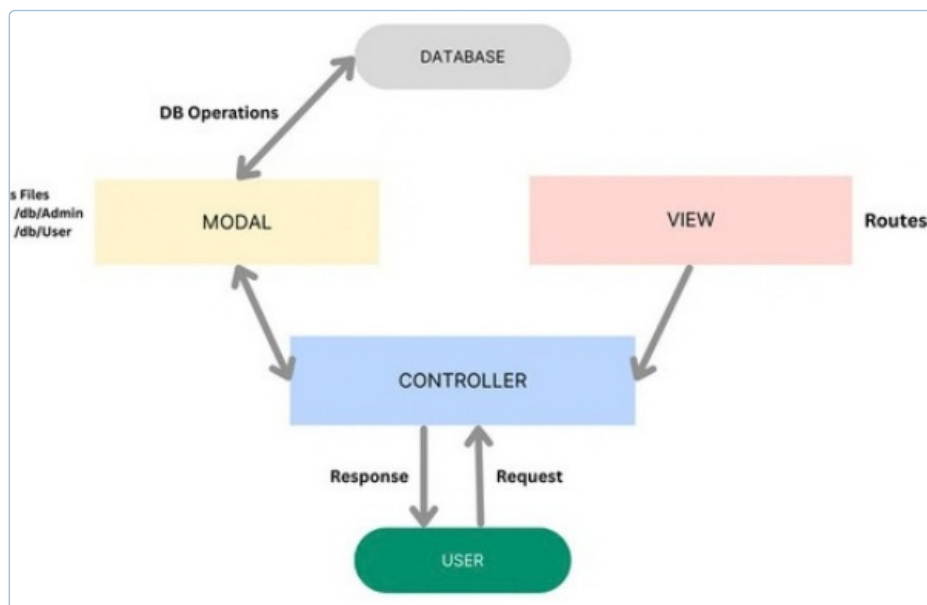


Fig. 4 – Model (Mongoose schemas over MongoDB) holds data and validation; Controller is the intermediary that captures requests, delegates to models or the Gemini service, and packages JSON output; the JSON response itself serves as the "View" for this headless API project. (The team's original diagram labels the Model box "MODAL" – the correct term used throughout this documentation is **Model**.)

EPIC 2 - Project Setup

Created the `FAQ` project folder with a `src` subfolder, opened in VS Code, then ran `npm init -y` followed by `npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai`. Scaffolded `src/` with `config/`, `controllers/`, `middleware/`, `models/`, `routes/`, `services/` and `utils/`, alongside root-level `app.js`, `server.js` and `.env`. Configured `.env` with `PORT`, `MONGO_URI`, `JWT_SECRET` and `GEMINI_API_KEY`.

EPIC 3 - Backend Development

The backend is built around a small set of core scripts. `src/server.js` is the gateway server node: it defines an async `connectDB` that requires `./config/db` and awaits the connection, then `startServer` awaits `connectDB()` before calling `app.listen` on `process.env.PORT || 5000`, with a `process.on('unhandledRejection', ...)` handler that logs the error and closes the server before exiting.

```

1  const app = require('./app');
2  const connectDB = async () => {
3    try {
4      const dbConnect = require('./config/db');
5      await dbConnect();
6    } catch (error) {
7      console.error('Failed to connect to the database', error);
8    }
9  };
10
11 const startServer = async () => {
12   // connect to database
13   await connectDB();
14
15   const PORT = process.env.PORT || 5000;
16
17   const server = app.listen(PORT, () => {
18     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19   });
20
21   // Handle unhandled promise rejections
22   process.on('unhandledRejection', (err, promise) => {
23     console.error('Unhandled Rejection: ${err.message}');
24     // Close server & exit process
25     server.close(() => process.exit(1));
26   });
27 };
28
29 startServer();
30

```

`src/server.js` – database connection bootstrap, server startup, and unhandled-rejection safety net

`src/app.js` is the gateway Express setup: it configures `cors()`, loads `dotenv`, wires in `errorHandler` from `middleware/errorMiddleware.js`, applies `express.json()` and `urlencoded` body parsing, exposes a `GET / status` route reporting `{ success: true, message: "AI FAQ Assistant API is running.", version: "1.0.0" }`, and mounts `/api/auth`, `/api/faqs` and `/api/ai`.

```

src > app.js >
1 const express = require('express');
2 const cors = require('cors');
3 const dotenv = require('dotenv');
4 const { errorHandler } = require('../middleware/errorMiddleware');
5
6 // load environment variables
7 dotenv.config();
8
9 const app = express();
10
11 // Inable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('../routes/authRoutes'));
29 app.use('/api/faqs', require('../routes/faqRoutes'));
30 app.use('/api/ai', require('../routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts websocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36

```

src/app.js – Express app setup, health-check route, and the three mounted API route groups

src/middleware/authMiddleware.js exports `protect`: it reads the `Authorization` header, confirms it starts with `Bearer`, verifies the token with `jwt.verify` against `process.env.JWT_SECRET`, and attaches `req.user` via `User.findById(decoded.id).select('-password')` – returning 401 if the token is missing, invalid, or the user no longer exists.

```

src/middleware > authMiddleware.js >
1 const jwt = require('jsonwebtoken');
2 const User = require('../models/User');
3
4 const protect = async (req, res, next) => {
5   let token;
6
7   if (
8     req.headers.authorization &&
9     req.headers.authorization.startsWith('Bearer')
10  ) {
11    try {
12      // Get token from header
13      token = req.headers.authorization.split(' ')[1];
14
15      // Verify token
16      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_long_and_random');
17
18      // Get user from the token, exclude password
19      req.user = await User.findById(decoded.id).select('-password');
20
21      if (!req.user) {
22        return res.status(401).json({
23          success: false,
24          message: 'Not authorized, user not found',
25        });
26      }
27
28      next();
29    } catch (error) {
30      console.error(error);
31      return res.status(401).json({
32        success: false,
33        message: 'Not authorized, token failed',
34      });
35    }
36  }
37 }
38

```

src/middleware/authMiddleware.js – Bearer-token verification and req.user attachment

src/middleware/errorMiddleware.js exports `errorHandler`: it starts from `res.statusCode` (or 500), respects a status already set on the error object (e.g. a 429 the Gemini SDK may set), and maps Mongoose duplicate-key errors (code 11000) to a 400 "Duplicate field value entered" message, `ValidationError` to a 400 with the joined field messages, and `CastError` (a malformed ObjectId) to a 404 – logging the stack trace to the console only for genuine 500s.

```

src/middleware > errorMiddleware.js >
1 const errorHandler = (err, req, res, next) => {
2   let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
3   let message = err.message;
4
5   // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
6   if (err.status) {
7     statusCode = err.status;
8   }
9
10  // Mongoose duplicate key error (e.g. duplicate email)
11  if (err.code === 11000) {
12    statusCode = 400;
13    const field = Object.keys(err.keyValue)[0];
14    message = `Duplicate field value entered: ${field}. Please use another value.`;
15  }
16
17  // Mongoose validation error
18  if (err.name === 'ValidationError') {
19    statusCode = 400;
20    message = Object.values(err.errors)
21      .map((val) => val.message)
22      .join(', ');
23  }
24
25  // Mongoose bad ObjectId
26  if (err.name === 'CastError') {
27    statusCode = 404;
28    message = `Resource not found with id of ${err.value}`;
29  }
30
31  // Only log stack trace to the console for unexpected 500 Server Errors
32  if (statusCode === 500) {
33    console.error(err.stack);
34  } else {
35

```

src/middleware/errorMiddleware.js – centralized error classification and sanitized JSON responses

MongoDB is the primary NoSQL datastore, accessed through Mongoose for schema validation and structured querying.

```
src > models > User.js > ...
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcrypt');
3
4 const userSchema = new mongoose.Schema({
5   {
6     name: {
7       type: String,
8       required: [true, 'Name is required'],
9       trim: true,
10    },
11    email: {
12      type: String,
13      required: [true, 'Email is required'],
14      unique: true,
15      lowercase: true,
16      trim: true,
17      match: [/^\s*@\s*\.\s*$/, 'Please provide a valid email address'],
18    },
19    passwords: {
20      type: String,
21      required: [true, 'Password is required'],
22      minlength: [6, 'Password must be at least 6 characters long'],
23    },
24  },
25  timestamps: true,
26 });
27
28 // Pre-save hook to hash the password before saving
29 userschema.pre('save', async function () {
30   if (!this.isModified('password')) {
31     return;
32   }
33   const salt = await bcrypt.genSalt(10);
34   this.password = await bcrypt.hash(this.password, salt);
35 });
```

src/models/User.js - name/email/password fields with a bcrypt pre('save') hook that salts (rounds = 10) and hashes the password before it is ever persisted

```
src > models > FAQ.js > ...
1 const mongoose = require('mongoose');
2
3 const faqSchema = new mongoose.Schema({
4   {
5     question: {
6       type: String,
7       required: [true, 'Question is required'],
8       trim: true,
9     },
10    answer: {
11      type: String,
12      required: [true, 'Answer is required'],
13      trim: true,
14    },
15    category: {
16      type: String,
17      required: [true, 'Category is required'],
18      enum: {
19        values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20        message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking, ...
21      },
22      default: 'General',
23    },
24    createdBy: {
25      type: mongoose.Schema.Types.ObjectId,
26      ref: 'User',
27      required: [true, 'Creator user reference is required'],
28    },
29  },
30  timestamps: true,
31 });
32
33 const FAQ = mongoose.model('FAQ', faqSchema);
```

src/models/FAQ.js - question/answer/category (validated enum, default "General") and a createdBy reference back to the creating User

Database Collections

Collection	Stores
Users	Account and authentication details for every registered creator
FAQs	Every FAQ's question, answer, category and creator reference

API Router Interface Endpoints

```
src > routes > authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    registerUser,
5    loginUser,
6    getUserProfile,
7  } = require('../controllers/authController');
8  const { protect } = require('../middleware/authMiddleware');
9  const {
10    validateRegister,
11    validateLogin,
12  } = require('../middleware/validationMiddleware');
13
14  // Public routes
15  router.post('/register', validateRegister, registerUser);
16  router.post('/login', validateLogin, loginUser);
17
18  // Protected routes
19  router.get('/profile', protect, getUserProfile);
20
21  module.exports = router;
22
```

src/routes/authRoutes.js - public /register and /login , protected /profile

```
src > routes > faqRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    createFAQ,
5    getAllFAQs,
6    getFAQById,
7    updateFAQ,
8    deleteFAQ,
9    searchFAQs,
10  } = require('../controllers/faqController');
11  const { protect } = require('../middleware/authMiddleware');
12  const { validateFAQ } = require('../middleware/validationMiddleware');
13
14  // Public routes - GET operations
15  router.get('/search', searchFAQs); // Note: defined before /:id to prevent routing issues
16  router.get('/', getAllFAQs);
17  router.get('/:id', getFAQById);
18
19  // Protected routes - Modification operations
20  router.post('/', protect, validateFAQ, createFAQ);
21  router.put('/:id', protect, updateFAQ);
22  router.delete('/:id', protect, deleteFAQ);
23
24  module.exports = router;
25
```

src/routes/faqRoutes.js - public /search (declared before /:id), public list/get-by-id, protected create/update/delete

```
src > routes > aiRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    generateAIAnswer,
5    generateAIFaq,
6  } = require('../controllers/aiController');
7  const { protect } = require('../middleware/authMiddleware');
8  const {
9    validateAIAnswer,
10    validateAIFaq,
11  } = require('../middleware/validationMiddleware');
12
13  // Protected routes to prevent API overuse/abuse
14  router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15  router.post('/generate-faq', protect, validateAIFaq, generateAIFaq);
16
17  module.exports = router;
18
```

src/routes/aiRoutes.js - protected /answer and /generate-faq , both guarded against API overuse

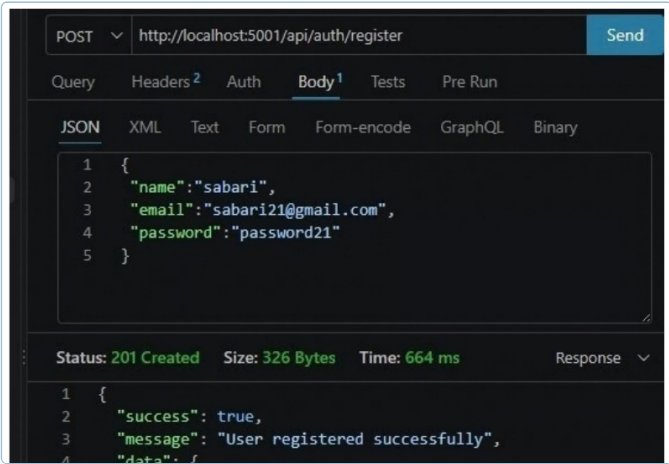
EPIC 5 - Execution & API Testing Evidence

API Route Interface Endpoints

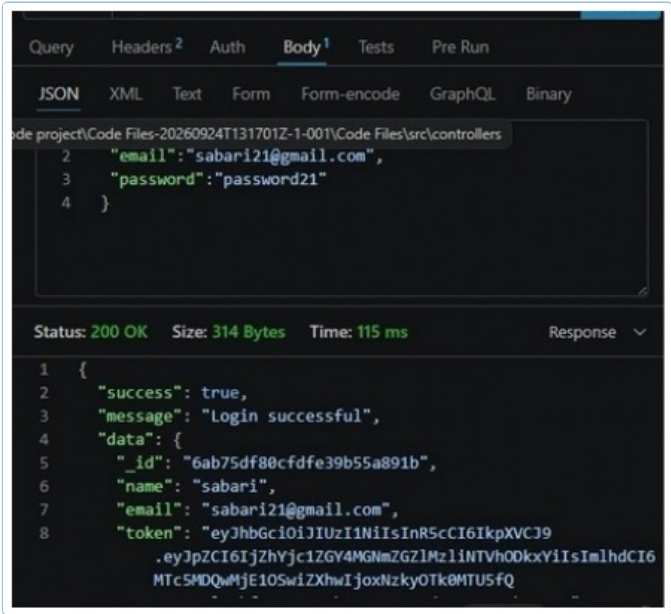
Group	Endpoint	Method	Access	Purpose
Authentication	/api/auth/register	POST	Public	Registers a new user and returns a JWT
	/api/auth/login	POST	Public	Authenticates a registered user and returns a fresh JWT

	/api/auth/profile	GET	Private	Returns the authenticated user's own profile
FAQ	/api/faqs	POST	Private	Creates a new FAQ under the authenticated creator
	/api/faqs	GET	Public	Lists all FAQs
	/api/faqs/search?q=...	GET	Public	Keyword search across FAQs
	/api/faqs/:id	GET	Public	Retrieves a single FAQ by ID
	/api/faqs/:id	PUT / DELETE	Private	Updates or deletes an FAQ
AI	/api/ai/answer	POST	Private	Generates a direct AI answer to a support question
	/api/ai/generate-faq	POST	Private	Generates a draft FAQ question/answer pair from a topic

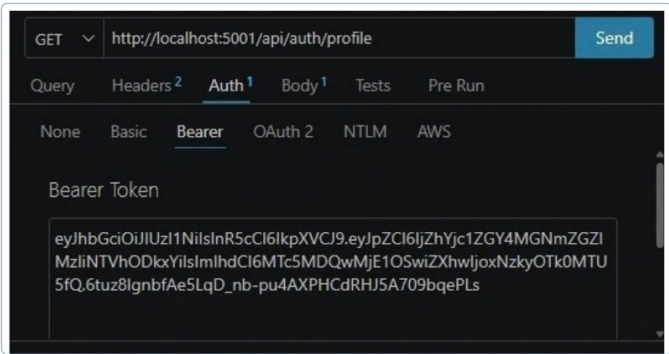
Test Evidence



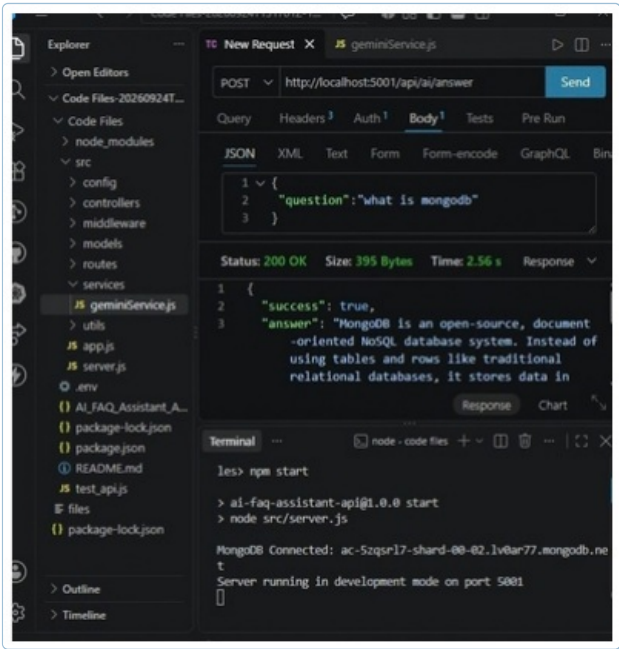
Register (user "sabari") - 201 Created, 326 Bytes, 664 ms



Login (user "sabari") - 200 OK, 314 Bytes, 115 ms



Get Profile - 200 OK, 232 Bytes, 48 ms



AI Answer ("what is mongodb") - 200 OK, 395 Bytes, 2.56 s

The create-FAQ, list/search-FAQ and generate-faq calls were exercised with the real request payloads shown in the team's write-up but were not captured with an individual response-time screenshot - see Performance Testing.

Demo & Repository	
Resource	Link
GitHub Repository	https://github.com/sabarivasan-2105/AI_FAQ_Assistant_API
Demo / API Testing Video	Google Drive recording

Conclusion

The AI FAQ Assistant API delivers a secure, JWT-and-bcrypt-protected FAQ management backend within the 15-29 September 2026 cycle: full CRUD and keyword search over categorized FAQ records backed by MongoDB, plus two Google Gemini-powered endpoints that either answer a support question directly or draft a new FAQ pair from a topic - directly resolving the disorganized, manually-drafted, hard-to-search documentation experience identified in the team's initial problem statements.

